

UpMyRank — PTB-Informed Implementation Roadmap

Immediate Pre-Work (Before Any Phase)

Two things that block everything else:

Run the DB migration — `migrate_v4_memory.sql` must be applied before any memory feature works. Do this now.

Fix hardcoded TEST_STUDENT_ID — every phase below assumes real student identity. Auth doesn't need to be full OAuth yet — even a simple name+email signup with Supabase is enough to unblock multi-student testing.

Phase 1 — Stabilize the Foundation

Goal: Close the open gaps from previous work. Ship a clean, stable base before adding new architecture.

What to build:

1a. Vision AI (7 items — already scoped in MEMORY.md) Complete as planned. `doubt.py` model validator, `_extract_question_from_image()` via GPT-4o, frontend upload UX, image thumbnail in student bubble, `handleSend` signature update. This is fully spec'ed — just needs execution.

1b. Confidence → Mastery Pipeline Send confidence level (Low/Medium/High) from frontend to backend on every hint call. Store in `doubt_blocks.student_confidence`. Update mastery scoring: high confidence + wrong = larger EMA penalty (misconception signal, not knowledge gap). High confidence + correct = larger EMA boost. Surface confidence badge in chat UI (already partially there).

1c. Frontend Error Boundaries Wrap chat UI in React error boundary. On API failure: show retry button, preserve chat history in state so student doesn't lose their session. Replace bare `Error: ...` text with a recovery UI.

1d. Basic Auth Supabase Auth — email/password signup only. Replace `TEST_STUDENT_ID` with real `user.id` from session. This unblocks multi-student testing and makes every subsequent phase meaningful.

Output of Phase 1: A stable, deployable product with real students, Vision AI, and confidence-aware mastery.

Phase 2 — Student Model Schema + Policy Engine

Goal: Implement the PTB's core architectural insight. Stop treating every student identically.

What to build:

2a. Student Model Schema Add `(persona_profile JSONB)` to `(student_memory)` table (via `migrate_v5_persona.sql`). Fields:

```
json
{
  "scaffolding_level": "HIGH | MEDIUM | LOW",
  "preferred_style": "analogy | formula | example | visual",
  "common_misconceptions": [],
  "allowed_hint_depth": 3,
  "interaction_depth_score": 0.0,
  "learning_velocity": 0.0
}
```

Initialize with defaults on student creation. Update `(scaffolding_level)` after every 5 sessions based on performance pattern. Update `(preferred_style)` from confidence + error data over time.

2b. Policy Engine Create `(app/services/policy/engine.py)`. Single function: `(select_pedagogy(student_model, topic, hint_level) → PedagogyConfig)`. Output determines: max concepts per response, use analogies (bool), information density (low/high), Socratic question style (conceptual/formula/application).

Pipeline change:

```
Before: query → RAG → LLM
After:  query → student_model → policy_selection → RAG → LLM with constraints
```

2c. Separate Customization from Personalization in prompts Split `(TUTOR_SYSTEM_PROMPT)` into two blocks:

- `(CUSTOMIZATION_PROMPT)` — global rules, never changes per student (Socratic rules, out-of-scope, LaTeX formatting, safety fallback). Set once.
- `(PERSONALIZATION_PROMPT)` — injected dynamically per student (scaffolding level, preferred style, misconception flags from student model).

2d. Cognitive Load Chunking Add to Policy Engine output: `(max_concepts_per_response = 3)` for HIGH scaffolding, `(5)` for LOW. Inject into prompt: "Explain at most {N} concepts in this response. If more are needed, end with a check-in question." Add confirmation pacing at hint_level 1: end every response with one check-in before progressing.

Output of Phase 2: The system now teaches differently based on who the student is, not just what they asked.

Phase 3 — Misconception Detection

Goal: Move from "correcting answers" to "correcting thinking."

What to build:

3a. Misconception Library Create `app/services/doubt/misconceptions.py` — a structured table of common JEE Physics wrong mental models. Format per entry:

```
python

{
  "pattern": regex or semantic trigger,
  "misconception": "Centripetal force is a real force acting outward",
  "correction_prompt": "Before we continue — what TYPE of force is centripetal? Is it",
  "concepts_affected": ["circular_motion", "newtons_laws"]
}
```

Start with 30 entries covering the highest-frequency JEE Physics misconceptions (sign errors in rotation, confusing velocity and acceleration in circular motion, work done by normal force, etc.).

3b. Misconception Check Step In `engine.py`, before generating a hint response, run a fast check: does the student's response match any misconception pattern? If yes → override normal hint generation with the correction prompt. Tag the event in `session_events` with `misconception_detected = true` and the misconception ID. Update `student_memory.persona_profile.common_misconceptions`.

3c. Misconception-aware mastery update When a misconception is detected: apply larger mastery penalty than a normal wrong answer. Flag the concept in `error_fingerprint` with `type: misconception` (vs `type: calculation_error`). These need different remediation — misconceptions require conceptual re-teaching, not just more practice.

Output of Phase 3: The system catches and corrects wrong mental models, not just wrong answers.

Phase 4 — Golden Dataset + Judge LLM

Goal: Make quality measurable. Stop guessing whether the Socratic engine is actually working.

What to build:

4a. Golden Dataset (50 triplets) Create `data/golden_dataset.json`. Structure per entry:

```
json
{
  "id": "gd_001",
  "context_chunk": "NCERT source text",
  "student_question": "...",
  "ideal_socratic_response": "...",
  "hint_level": 0,
  "topic": "rotational_dynamics",
  "misconception_addressed": null | "misconception_id"
}
```

Process: use GPT-4o to draft ideal responses for 50 common JEE Physics questions → manually review and edit each one → commit to Git as versioned ground truth. This is a **data task**, not a code task. Do it in parallel with other phases.

4b. Judge LLM Create `app/services/eval/judge.py`. After every hint_level 0-2 response, async fire a Judge LLM call (gpt-4o-mini, non-blocking):

```
Rubric:
score 0 = gave full solution or direct answer
score 1 = gave a hint but too vague, no thought prompted
score 2 = asked a leading Socratic question that forces reasoning
```

Store result in `session_events.scaffolding_score`. Never block the student response on this — fire and forget, collect data.

4c. Pedagogy Drift Detection Weekly batch job: query all sessions from last 7 days → calculate `avg_scaffolding_score` per topic → flag topics where score < 1.5 (pedagogy drift) → generate report → use to update `CUSTOMIZATION_PROMPT` or golden dataset. This is the continuous eval loop.

Output of Phase 4: You can now prove the system is working pedagogically, not just technically. Promotable claim: "Our Socratic adherence rate is X%."

Phase 5 — Evaluation Dashboard + Regression Gate

Goal: Make the system self-monitoring. Know when a deployment breaks something.

What to build:

5a. Metrics Pipeline Track and store per session: `scaffolding_score`, `misconception_detected`, `retrieval_similarity_max`, `response_latency_ms`,

`hint_was_useful` (inferred: did student solve after this hint?), `out_of_scope_refusal_fired`.

5b. RAG Retrieval Confidence Threshold Log cosine similarity of top retrieved chunk on every query. If max similarity $< 0.5 \rightarrow$ flag as low-confidence retrieval in `session_events`. Currently retrieval failures are invisible — the LLM generates an answer regardless of whether the retrieved context was relevant.

5c. Live Status Dashboard New admin page `/admin` (protected route). Supabase views powering:

- P95 response latency (last 24h)
- Socratic Adherence Rate (avg scaffolding_score across sessions)
- Misconception Detection Rate
- Out-of-Scope Refusal Accuracy
- Low-confidence Retrieval Rate
- Mastery Gain per student per week

5d. Regression Gate Before any deployment: run 20 golden dataset queries through the live engine $\rightarrow$ compare scaffolding scores $\rightarrow$ if avg drops $> 10\%$ vs. previous baseline $\rightarrow$ block deployment. Automate as a GitHub Actions check.

Output of Phase 5: Every deployment is verified. The system cannot silently regress.

Phase 6 — Semantic Caching + Latency

Goal: Fix the flow state problem. Sub-2 second responses on repeated concepts.

What to build:

6a. Semantic Cache (Redis) Redis is already in the stack. Wire it: on every query, embed and check similarity against cached queries (threshold 0.92). Cache hit $\rightarrow$ return cached response instantly. Cache miss $\rightarrow$ run full pipeline $\rightarrow$ store result with 24hr TTL. Estimate: 40-60% cache hit rate on common Physics questions.

6b. Streaming text Implement SSE streaming on Socratic responses. Student sees first tokens within 300ms instead of waiting 2-3 seconds for full response. Dramatically reduces perceived latency and keeps flow state intact.

6c. Latency monitoring Add timing middleware to FastAPI. Log `response_latency_ms` on every request. Alert (log + email) if P95 $> 3000\text{ms}$. Feed into admin dashboard.

Output of Phase 6: The product feels fast. Students don't break flow state waiting for responses.

Phase Summary

Phase	Name	Claude Code Sessions	Prerequisite
Pre-work	DB migration + basic auth	1	—
1	Stabilize (Vision AI + Confidence + Error Boundaries)	2	Pre-work
2	Student Model + Policy Engine	2	Phase 1
3	Misconception Detection	1	Phase 2
4	Golden Dataset + Judge LLM	2 (+ data task)	Phase 3
5	Evaluation Dashboard	2	Phase 4
6	Semantic Cache + Streaming	1	Phase 5

What This Unlocks

After Phase 2: UpMyRank stops being "a well-prompted GPT wrapper" and becomes a system with a real Policy Engine. Demo-worthy claim: "The AI teaches differently based on who you are."

After Phase 3: "The AI detects when you have a wrong mental model and corrects your thinking, not just your answer."

After Phase 4: "Our Socratic Adherence Rate is 87%. We measure this daily."

After Phase 5: "Every deployment is regression-tested against a golden dataset before going live."

These are claims no competitor can currently make.